
SQL Training Strategy — Trainer Demo Framework

Recognize the problem pattern and construct the solution.

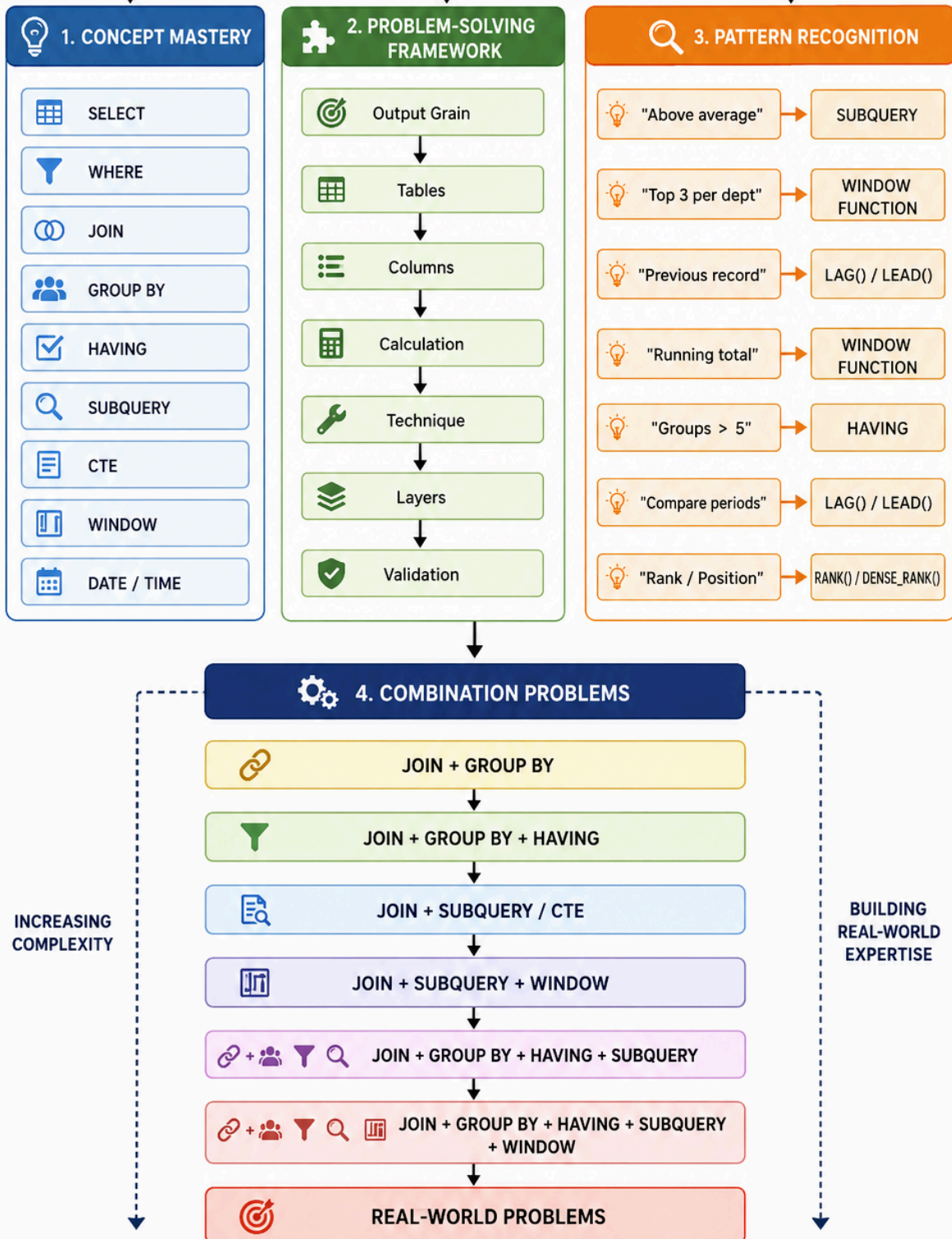
The ultimate skill should be:

Question → Pattern → Grain → Technique → Layers → SQL → Validation

1. The Overall SQL Learning Architecture

SQL LEARNING ARCHITECTURE

SQL PROBLEM SOLVING



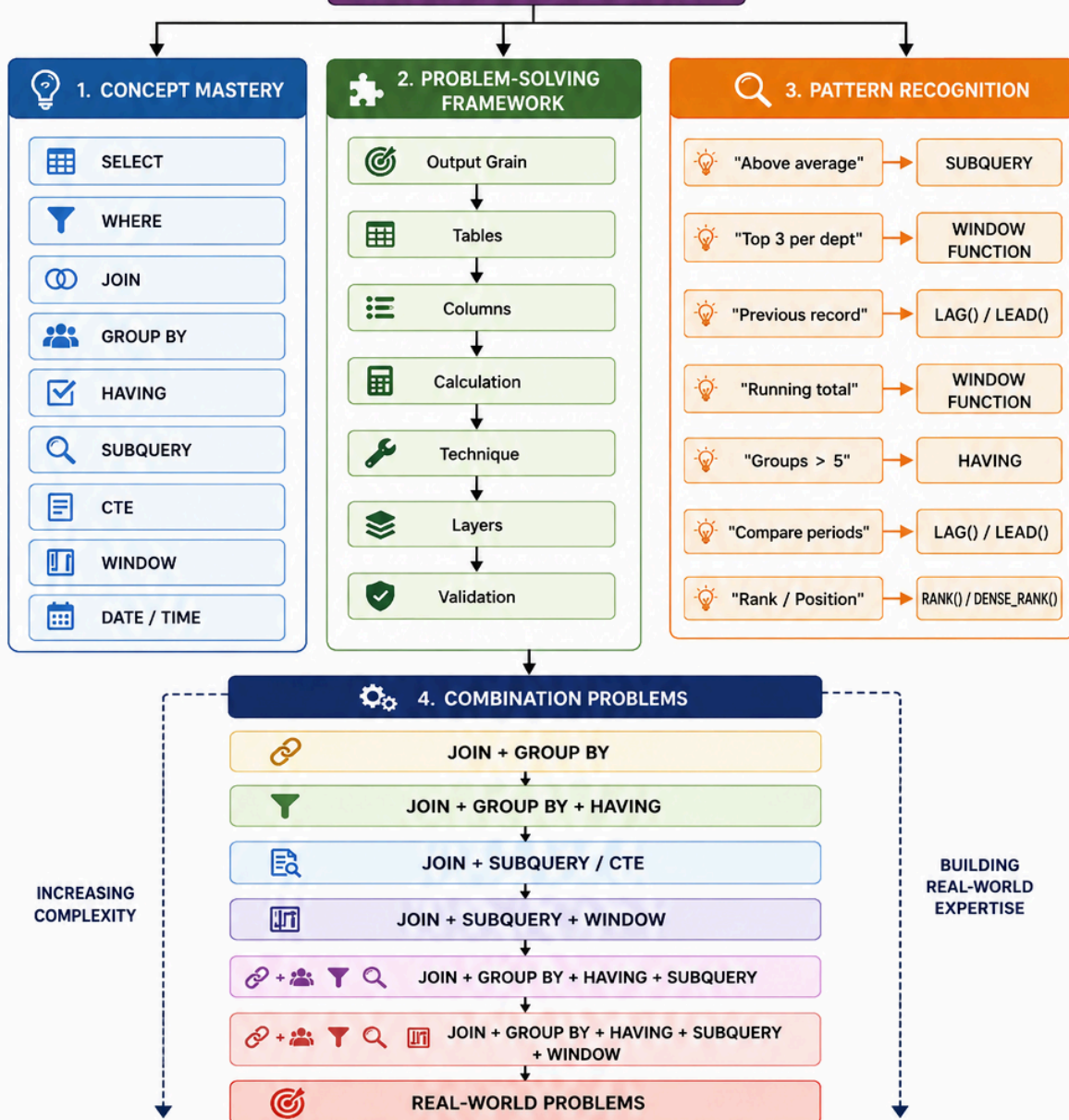
2. The Four Pillars

Pillar	Objective	What trainee should learn
1. Concept Mastery	Understand SQL functionality	What does JOIN, GROUP BY, HAVING, etc. do?
2. Problem-Solving Framework	Learn how to attack a problem	How do I break a question into pieces?
3. Combination Problems	Build complexity	How do multiple SQL techniques work together?
4. Assessment Simulation	Validate independence	Can I solve an unfamiliar problem without help?

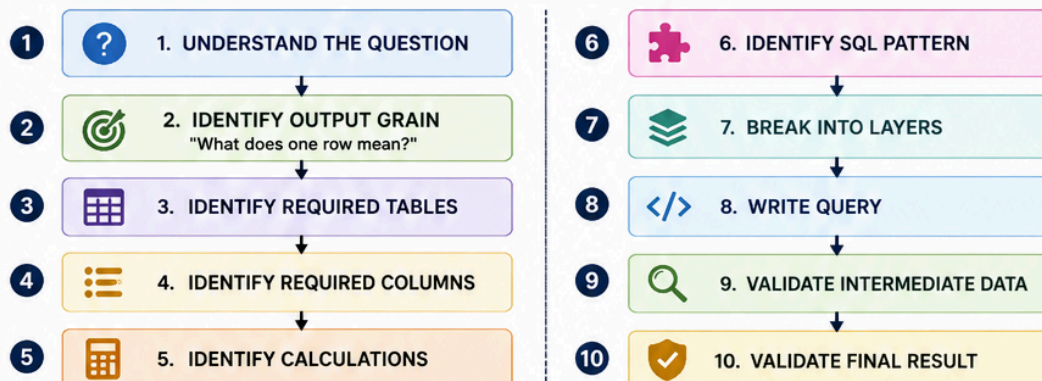
3. The Core Problem-Solving Framework

SQL LEARNING ARCHITECTURE

SQL PROBLEM SOLVING



SQL PROBLEM-SOLVING WORKFLOW (For Every SQL Problem Worksheet)



The most important question

Learn to ask:

“What does one row in my final output represent?”

That is **output grain**.

For example:

Question: Find total sales for each customer.

One output row =

ONE CUSTOMER

Therefore:

GROUP BY customer

But:

Question: Find monthly sales for each customer.

One output row =

ONE CUSTOMER + ONE MONTH

Therefore:

GROUP BY customer, month

And:

Question: Find the top 3 products in each category.

One output row =

ONE PRODUCT

within ONE CATEGORY

But now we need ranking **within category**:

PARTITION BY category

This is the kind of reasoning you want trainees to develop.

4. Pattern Dictionary

Give this to trainees **before you start complex SQL problems.**

Question wording / pattern	Think about	Likely technique
Highest salary	Maximum value	MAX()
Lowest salary	Minimum value	MIN()
Average salary	Average	AVG()
Total sales	Aggregation	SUM()
Number of employees	Counting	COUNT()
Customers with more than 5 orders	Aggregate condition	GROUP BY + HAVING
Above average salary	Compare with overall average	Subquery
Above departmental average	Compare with group average	Correlated subquery / window
Top 3 employees	Global ranking	ORDER BY + LIMIT / window
Top 3 per department	Ranking within group	ROW_NUMBER() / RANK()
Previous transaction	Previous row	LAG()
Next transaction	Next row	LEAD()
Running total	Cumulative calculation	SUM() OVER()
Ranking	Relative position	RANK() / DENSE_RANK()
Remove duplicates	Number rows	ROW_NUMBER()
Latest customer order	Latest row	ROW_NUMBER() / MAX()
First transaction	Earliest row	ROW_NUMBER() / MIN()
Same manager	Compare rows in same table	Self Join
Monthly sales	Group by month	GROUP BY + DATE
Previous month sales	Compare months	LAG()
Month-over-month growth	Current vs previous	LAG() + calculation
Customers who never ordered	Missing relationship	LEFT JOIN / NOT EXISTS
Products never sold	Missing relationship	LEFT JOIN / NOT EXISTS
Above departmental average	Group comparison	Window / subquery
Top N in each category	Partitioned ranking	Window function
Complex multi-stage problem	Multiple transformations	CTE / derived table
Reuse intermediate result	Named intermediate dataset	CTE
Filter aggregated result	Condition after grouping	HAVING

5. The SQL Training Ladder

I recommend this progression.

Level	Skills
Level 1	SELECT, WHERE
Level 2	JOIN
Level 3	GROUP BY, aggregates
Level 4	HAVING
Level 5	Subqueries
Level 6	CTE
Level 7	Window functions
Level 8	Date/time
Level 9	Combination problems
Level 10	Real-world SQL problems

6. Demo Database

Use one consistent business domain throughout training.

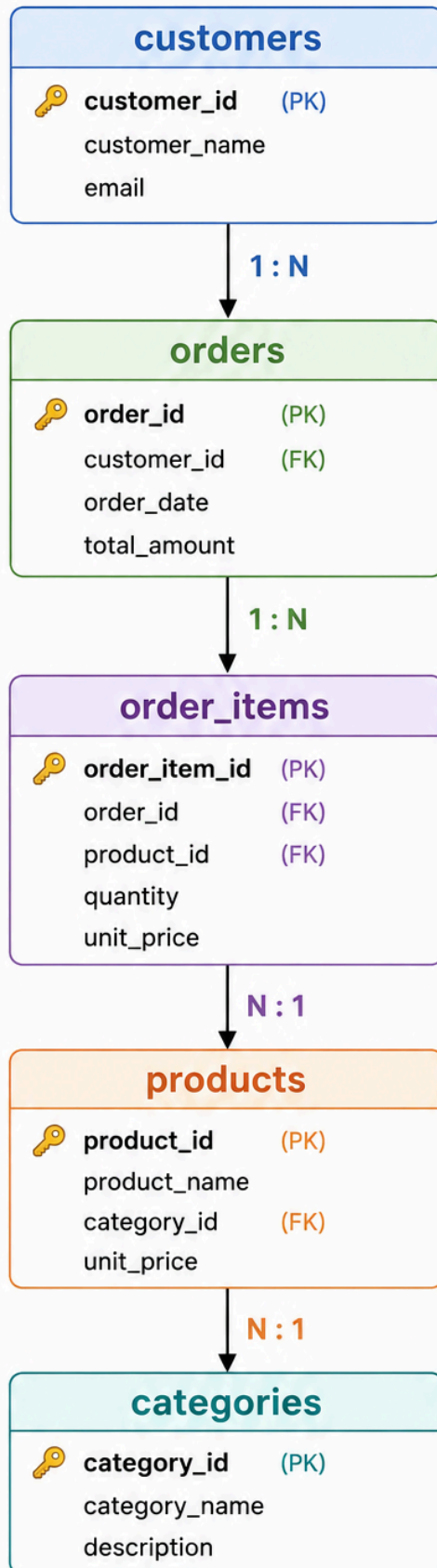
I recommend:

E-Commerce Sales Database

It is excellent because the same schema can support almost every SQL concept.



TABLES



7. Database Setup

Assume **MySQL 8+**.

Categories

```
CREATE TABLE categories (  
    category_id INT PRIMARY KEY,  
    category_name VARCHAR(50)  
);  
INSERT INTO categories VALUES  
(1, 'Electronics'),  
(2, 'Furniture'),  
(3, 'Books');
```

Products

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    category_id INT,  
    FOREIGN KEY (category_id)  
        REFERENCES categories(category_id)  
);  
INSERT INTO products VALUES  
(101, 'Laptop', 1),  
(102, 'Mouse', 1),  
(103, 'Keyboard', 1),  
(201, 'Chair', 2),  
(202, 'Desk', 2),  
(301, 'SQL Book', 3),  
(302, 'Python Book', 3);
```

Customers

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(50)  
);
```

```
INSERT INTO customers VALUES
```

```
(1, 'Arun', 'Chennai'),  
(2, 'Priya', 'Bangalore'),  
(3, 'Rahul', 'Mumbai'),  
(4, 'Sneha', 'Chennai'),  
(5, 'Kiran', 'Hyderabad');
```

Orders

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)
```

```
);
```

```
INSERT INTO orders VALUES
```

```
(1001, 1, '2026-01-05'),  
(1002, 2, '2026-01-10'),  
(1003, 1, '2026-02-05'),  
(1004, 3, '2026-02-15'),  
(1005, 2, '2026-03-01'),  
(1006, 4, '2026-03-10'),  
(1007, 1, '2026-03-20'),  
(1008, 3, '2026-04-05'),  
(1009, 5, '2026-04-15');
```

Order Items

```
CREATE TABLE order_items (  
  order_item_id INT PRIMARY KEY,  
  order_id INT,  
  product_id INT,  
  quantity INT,  
  unit_price DECIMAL(10,2),  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)
```

```
);
```

```
INSERT INTO order_items VALUES
```

```
(1, 1001, 101, 1, 80000),  
(2, 1001, 102, 2, 1000),  
(3, 1002, 201, 2, 5000),  
(4, 1003, 103, 1, 2500),  
(5, 1003, 301, 2, 800),
```

(6, 1004, 101, 1, 80000),
(7, 1005, 202, 1, 12000),
(8, 1006, 302, 3, 900),
(9, 1007, 102, 1, 1000),
(10, 1008, 201, 1, 5000),
(11, 1008, 301, 1, 800),
(12, 1009, 103, 2, 2500);

8. Topic-by-Topic Demo Problems

For each topic, use:

1 Simple + 1 Medium

The simple problem teaches the concept.

The medium problem teaches the **pattern**.

TOPIC 1 — SELECT

Simple

Problem

Display the name and city of every customer.

Pattern

Question



Need columns



No filtering



SELECT

Query

SELECT customer_name, city

FROM customers;

Grain

ONE ROW = ONE CUSTOMER

Medium

Problem

Display each product with its product name and category ID.

Thinking

Required table:

products

Required columns:

product_name

category_id

No join is required because **category_id** already exists in products.

```
SELECT product_name, category_id
FROM products;
```

Teaching point

Question:

"Do we need the categories table?"

Correct answer:

No, unless the question asks for **category_name**.

This teaches trainees not to join tables unnecessarily.

TOPIC 2 — WHERE

Simple

Problem

Find all customers from Chennai.

Pattern

Find records



Condition



WHERE

SELECT *

FROM customers

WHERE city = 'Chennai';

Medium

Problem

Find all orders placed after February 1, 2026.

SELECT *

FROM orders

WHERE order_date > '2026-02-01';

Pattern

Date condition



WHERE



Compare date

TOPIC 3 — JOIN

Simple

Problem

Display each order along with the customer name.

Identify tables

orders

Customers

```
select o.order_id , c.customer_name customer_name from orders o
Join Customers c on o.customer_id = c.customer_id;
```

Relationship

orders.customer_id

=

customers.customer_id

Query

```
SELECT
  o.order_id,
  c.customer_name,
  o.order_date
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id;
```

Pattern

Information exists in two tables

↓

JOIN

Medium

Problem

Display each order with customer name and the products purchased.

Select

O.order_id'

C.customer_name'

P.product_name

From orders as o

Join customers as c

On o.order_id=c.order_id

Join products as p

On p.product_id=c.product_id;

Now:

orders



customers

orders



order_items



products

Query

```
SELECT
  o.order_id,
  c.customer_name,
  p.product_name,
  oi.quantity
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id;
```

Teaching point

Ask:

"Why do we need four tables?"

Because the requested information is distributed across four tables.

TOPIC 4 — GROUP BY

Simple

Problem

Find the number of orders placed by each customer.

Output grain

ONE ROW = ONE CUSTOMER

Calculation

COUNT(order_id)

Query

```
SELECT
  customer_id,
  COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

Pattern

"for each customer"

↓

GROUP BY customer

This phrase is extremely important.

Medium

Problem

Find total quantity of products purchased by each customer.

Now we need:

customers
↓
orders
↓
order_items

Query

```
SELECT
  o.customer_id,
  SUM(oi.quantity) AS total_quantity
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY o.customer_id;
```

Pattern

For each customer
+
Total quantity
↓
GROUP BY
+
SUM()

TOPIC 5 — HAVING

Simple

Problem

Find customers who have placed more than 2 orders.

First solve aggregation

Then:

Need groups where count > 2



HAVING

Final:

```
SELECT  
  customer_id,  
  COUNT(*) AS order_count  
FROM orders  
GROUP BY customer_id  
HAVING COUNT(*) > 2;
```

Key teaching point

WHERE → filters rows

HAVING → filters groups

Medium

Problem

Find customers whose total purchase quantity is greater than 3.

Pattern:

JOIN



GROUP BY



SUM



Filter aggregate

↓
HAVING

TOPIC 6 — SUBQUERY

Simple

Problem

Find products whose product ID is greater than the average product ID.

This is mainly a demonstration of the subquery pattern.

```
SELECT *  
FROM products  
WHERE product_id >  
    (  
        SELECT AVG(product_id)  
        FROM products  
    );
```

Pattern

Compare value
with
calculated value
↓
SUBQUERY

Medium

Problem

Find customers who placed more orders than the average number of orders placed by customers.

This is a much better training problem.

Layer 1

Find orders per customer.

```
SELECT
  customer_id,
  COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

Layer 2

Find the average order count.

```
SELECT AVG(order_count)
FROM
(
  SELECT
    customer_id,
    COUNT(*) AS order_count
  FROM orders
  GROUP BY customer_id
) x;
```

Layer 3

Compare each customer's count.

```
SELECT
  customer_id,
  COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >
(
  SELECT AVG(order_count)
  FROM
  (
    SELECT
      customer_id,
      COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
  ) x
);
```

Teaching lesson

This is where learners should understand:

A subquery is often not a “trick”; it is simply another layer of calculation.

TOPIC 7 — CTE

Simple

Problem

Create a temporary result containing the number of orders per customer and display it.

```
WITH customer_orders AS
(
  SELECT
    customer_id,
    COUNT(*) AS order_count
  FROM orders
  GROUP BY customer_id
)
SELECT *
FROM customer_orders;
```

Medium

Problem

Find customers whose order count is greater than the average customer order count.

Layer 1

```
WITH customer_orders AS
(
  SELECT
    customer_id,
    COUNT(*) AS order_count
  FROM orders
```

```
GROUP BY customer_id  
)
```

Layer 2

```
WITH customer_orders AS  
(  
  SELECT  
    customer_id,  
    COUNT(*) AS order_count  
  FROM orders  
  GROUP BY customer_id  
)  
SELECT AVG(order_count)  
FROM customer_orders;
```

Final

```
WITH customer_orders AS  
(  
  SELECT  
    customer_id,  
    COUNT(*) AS order_count  
  FROM orders  
  GROUP BY customer_id  
)  
SELECT  
  customer_id,  
  order_count  
FROM customer_orders  
WHERE order_count >  
(  
  SELECT AVG(order_count)  
  FROM customer_orders  
)  
);
```

Teaching point

Compare:

SUBQUERY

↓

Nested logic

with:

CTE



Named layers



Better readability

TOPIC 8 — WINDOW FUNCTIONS

This is where your framework becomes especially valuable.

Simple

Problem

Display each customer and their order count along with the overall average order count.

First:

```
SELECT
  customer_id,
  COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

Then use a window function.

```
WITH customer_orders AS
(
  SELECT
    customer_id,
    COUNT(*) AS order_count
  FROM orders
  GROUP BY customer_id
)
SELECT
  customer_id,
  order_count,
  AVG(order_count) OVER () AS avg_order_count
FROM customer_orders;
```

Key concept

GROUP BY:

Many rows

↓

One row per group

Window function:

Rows remain

+

Calculation is added

Medium

Problem

Rank customers based on their total purchase amount.

Layer 1 — Calculate customer revenue

```
WITH customer_sales AS
(
  SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  GROUP BY o.customer_id
)
```

Layer 2 — Rank

```
SELECT
  customer_id,
  total_sales,
  RANK() OVER (
    ORDER BY total_sales DESC
  ) AS sales_rank
FROM customer_sales;
```

Pattern

Total sales
↓
GROUP BY customer
↓
Ranking
↓
RANK()

TOPIC 9 — DATE/TIME

Simple

Problem

Display each order and the month in which it was placed.

```
SELECT
  order_id,
  order_date,
  MONTH(order_date) AS order_month
FROM orders;
```

Medium

Problem

Find total sales for each month.

Required tables

orders
order_items

Revenue

quantity × unit_price

Month

MONTH(order_date)

Query

```
SELECT
  YEAR(o.order_date) AS order_year,
  MONTH(o.order_date) AS order_month,
  SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY
  YEAR(o.order_date),
  MONTH(o.order_date)
ORDER BY
  order_year,
  order_month;
```

9. Now Start Combination Problems

This is where trainees should stop thinking about individual SQL commands.

They should start thinking:

“What are the stages?”

Combination 1 — JOIN + GROUP BY

Simple

Find the total sales generated by each customer.

Pattern:

```
Orders
+
Order Items
↓
JOIN
↓
Revenue
```

↓
GROUP BY customer
SELECT
 o.customer_id,
 SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
 ON o.order_id = oi.order_id
GROUP BY o.customer_id;

Combination 2 — JOIN + GROUP BY + HAVING

Medium

Find customers whose total sales exceed ₹10,000.

```
SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY o.customer_id
HAVING SUM(oi.quantity * oi.unit_price) > 10000;
```

Pattern:

```
JOIN
↓
Calculate revenue
↓
GROUP BY customer
↓
HAVING
```

Combination 3 — JOIN + SUBQUERY

Medium

Find customers whose total spending is above the average customer spending.

Layer 1

```
SELECT
  o.customer_id,
  SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY o.customer_id;
```

Layer 2

Calculate average:

```
SELECT AVG(total_sales)
FROM
(
  SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  GROUP BY o.customer_id
) x;
```

Final

```
SELECT
  o.customer_id,
  SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY o.customer_id
HAVING SUM(oi.quantity * oi.unit_price) >
(
  SELECT AVG(total_sales)
  FROM
  (
    SELECT
      o2.customer_id,
```

```
SUM(oi2.quantity * oi2.unit_price) AS total_sales
FROM orders o2
JOIN order_items oi2
  ON o2.order_id = oi2.order_id
GROUP BY o2.customer_id
) x
);
```

10. Important Demonstration

Question:

Find the top 2 products by revenue for every category.

Do **not** show SQL immediately.

Ask these questions one at a time.

Question 1

What is being asked?

Top 2 products

Question 2

Top 2 based on what?

Revenue

Question 3

For what?

Every category

Question 4

What does one output row represent?

ONE PRODUCT
within ONE CATEGORY

Question 5

Which tables do we need?

products
categories
order_items

Potentially:

products
↓
order_items

Category comes from:

products.category_id

Question 6

How do we calculate revenue?

quantity × unit_price

Question 7

How do we get product-level revenue?

GROUP BY product

Question 8

How do we identify the top 2 within each category?

This is the key recognition moment:

Top N
+
For every category
↓
WINDOW FUNCTION

Specifically:

ROW_NUMBER()
OVER (

```
PARTITION BY category
ORDER BY revenue DESC
)
```

11. Build the Problem in Layers

Layer 1 — Product Revenue

```
SELECT
  p.product_id,
  p.product_name,
  p.category_id,
  SUM(oi.quantity * oi.unit_price) AS revenue
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY
  p.product_id,
  p.product_name,
  p.category_id;
```

Stop.

Show the intermediate result.

Ask:

"Does this result correctly represent one row per product?"

If yes, continue.

Layer 2 — Add Ranking

```
SELECT
  product_id,
  product_name,
  category_id,
  revenue,
  ROW_NUMBER() OVER (
```

```

        PARTITION BY category_id
        ORDER BY revenue DESC
    ) AS product_rank
FROM
(
    SELECT
        p.product_id,
        p.product_name,
        p.category_id,
        SUM(oi.quantity * oi.unit_price) AS revenue
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    GROUP BY
        p.product_id,
        p.product_name,
        p.category_id
) AS product_sales;

```

Layer 2 — Add Ranking (WITH CTE)

```

WITH product_sales AS
(
    SELECT
        p.product_id,
        p.product_name,
        p.category_id,
        SUM(oi.quantity * oi.unit_price) AS revenue
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    GROUP BY
        p.product_id,
        p.product_name,
        p.category_id
)
SELECT
    product_id,
    product_name,
    category_id,
    revenue,

```



```

ROW_NUMBER() OVER
(
  PARTITION BY category_id
  ORDER BY revenue DESC
) AS product_rank
FROM product_sales;

```

Now we have:

Category	Product	Revenue	Rank
----------	---------	---------	------

1	Laptop	80000	1
1	Mouse	3000	2
1	Keyboard	7500	...
2	Desk	12000	1
2	Chair	15000	2

Layer 3 — Filter Top 2

Because a window function cannot normally be filtered directly in the same **WHERE** clause where it is created, create another layer.

```

WITH product_sales AS
(
  SELECT
    p.product_id,
    p.product_name,
    p.category_id,
    SUM(oi.quantity * oi.unit_price) AS revenue
  FROM products p
  JOIN order_items oi
    ON p.product_id = oi.product_id
  GROUP BY
    p.product_id,
    p.product_name,
    p.category_id
),
ranked_products AS
(
  SELECT
    product_id,
    product_name,

```

```
category_id,  
revenue,  
ROW_NUMBER() OVER  
(  
    PARTITION BY category_id  
    ORDER BY revenue DESC  
) AS product_rank  
FROM product_sales  
)  
SELECT *  
FROM ranked_products  
WHERE product_rank <= 2;
```

12. This Is the Pattern You Want to Recognize

"Top 2"
↓
RANKING
↓
"For every category"
↓
PARTITION BY
↓
"Based on revenue"
↓
ORDER BY revenue DESC
↓
ROW_NUMBER()
↓
Filter rank <= 2

This is **much more valuable** than memorizing:

```
ROW_NUMBER() OVER (...)
```

13. Your Advanced Demo Problem

Now try this:

Find the top 2 products by revenue for every category for each month and compare their revenue with the previous month. Calculate the percentage change from the previous month.

This is an excellent **milestone-level problem**.

Don't solve it immediately.

First try create the pattern map.

QUESTION

↓
Top 2 products
↓
Revenue
↓
Every category
↓
Every month
↓
Previous month
↓
Percentage change

Now convert each phrase into SQL functionality:

Business phrase	SQL requirement
Product revenue	SUM(quantity * unit_price)
Every month	Date transformation
Every category	PARTITION BY category
Top 2	ROW_NUMBER()
Previous month	LAG()
Percentage change	Arithmetic calculation
Multiple stages	CTEs

So the trainee's plan should become:

STEP 1

Calculate product-month revenue

↓

STEP 2

Rank products within category + month

↓

STEP 3

Keep top 2



STEP 4

Use LAG() to get previous month revenue



STEP 5

Calculate percentage change



STEP 6

Validate

This is the **real SQL problem-solving skill**.

14. The Universal Question → Pattern → SQL Map

If the question says...	Ask yourself...	Likely solution
each customer	What is my group?	GROUP BY
total	What am I adding?	SUM()
average	What am I averaging?	AVG()
number of	What am I counting?	COUNT()
more than X	Am I filtering groups?	HAVING
above average	What am I comparing against?	Subquery / CTE
highest	Do I need maximum?	MAX() / ranking
top N	Global or within group?	LIMIT / window
top N per group	Ranking within group?	Window
previous	Previous row?	LAG()
next	Next row?	LEAD()
running	Cumulative?	SUM() OVER()
rank	Relative position?	RANK()
dense rank	Ranking without gaps?	DENSE_RANK()
latest	Most recent row?	ROW_NUMBER()
first	Earliest row?	ROW_NUMBER()
same	Compare rows?	Self Join

never	Missing relationship?	LEFT JOIN / NOT EXISTS
monthly	Date grouping?	Date functions
month-over-month	Previous period?	LAG()
percentage change	Current vs previous	Arithmetic + LAG()
multiple stages	Intermediate datasets	CTE

15. The "One Row Represents What?" Exercise

This deserves its **own training activity**.

Give trainees only the questions.

Question A

Find total sales for each customer.

Expected:

ONE ROW = ONE CUSTOMER

Question B

Find total sales for each customer for each month.

Expected:

ONE ROW = ONE CUSTOMER + ONE MONTH

Question C

Find the top 3 products in each category.

Expected:

ONE ROW = ONE PRODUCT + ONE CATEGORY

Question D

Find the monthly revenue and previous month's revenue for every product.

Expected:

ONE ROW = ONE PRODUCT + ONE MONTH

Question E

Find the top 2 products in every category for every month.

Expected:

ONE ROW = ONE PRODUCT

+

ONE CATEGORY

+

ONE MONTH

This exercise will dramatically improve their ability to choose:

GROUP BY

PARTITION BY

ORDER BY

16. GROUP BY vs PARTITION BY — Demo Illustration

Use this visual explanation.

GROUP BY

10 rows

↓

GROUP BY department

↓

3 rows

Department	Total
------------	-------

IT	500
----	-----

HR	300
----	-----

Sales 700

The original rows are **collapsed**.

Whereas:

WINDOW FUNCTION

10 rows

↓

PARTITION BY department

↓

10 rows remain

+

calculation added

Example:

Employee	Dept	Salary	Dept Avg
----------	------	--------	----------

A	IT	100	150
B	IT	200	150
C	HR	300	350
D	HR	400	350

This distinction should be drilled repeatedly.

17. Assessment Structure

For each module:

10 Easy

↓

10 Medium

↓

5 Hard

↓

5 Milestone

↓

1 Mock Assessment

Easy

Learner recognizes **one pattern**.

WHERE

JOIN

GROUP BY

Medium

Learner recognizes **two patterns**.

JOIN + GROUP BY

GROUP BY + HAVING

JOIN + SUBQUERY

Hard

Learner recognizes **three or more patterns**.

JOIN

+

GROUP BY

+

HAVING

+

SUBQUERY

Milestone

Learner must discover the pattern independently.

Business Question



No hints



Identify grain



Identify pattern



Build layers



SQL

18. Weekly Training Structure

Week 1 — Foundations

SELECT
WHERE
ORDER BY
DISTINCT
LIMIT

Practice:

10 Easy
10 Medium
5 Hard

Week 2 — JOIN

INNER JOIN
LEFT JOIN
Multiple JOINS
Self JOIN

Practice:

10 Easy
10 Medium
5 Hard

Week 3 — Aggregation

COUNT
SUM
AVG
MIN
MAX
GROUP BY
HAVING

Practice:

10 Easy
10 Medium
5 Hard

Week 4 — Subquery + CTE

Scalar subquery
IN
EXISTS
Correlated subquery
Derived table
CTE

Practice:

10 Easy
10 Medium
5 Hard
5 Milestone

Week 5 — Window Functions

ROW_NUMBER
RANK
DENSE_RANK
LAG
LEAD
SUM OVER
AVG OVER
PARTITION BY

Practice:

10 Easy
10 Medium
5 Hard
5 Milestone

Week 6 — Date/Time

YEAR
MONTH
DATE
DATEDIFF
DATE_ADD
DATE_SUB
Monthly aggregation
Period comparison

Week 7 — Combination

JOIN + GROUP BY
JOIN + GROUP BY + HAVING
JOIN + SUBQUERY
JOIN + CTE
JOIN + WINDOW

Week 8 — Real-World SQL

Problems such as:

Customer retention
Product performance
Monthly revenue
Top products
Repeat customers
Inactive customers
Customer ranking
Month-over-month growth
Latest transaction
Duplicate detection

19. The Problem-Solving Worksheet

Every trainee should receive the same worksheet.

=====
SQL PROBLEM WORKSHEET
=====

PROBLEM:

1. WHAT IS THE QUESTION ASKING?

2. WHAT DOES ONE OUTPUT ROW REPRESENT?

OUTPUT GRAIN:

3. WHICH TABLES ARE REQUIRED?

TABLE 1: _____

TABLE 2: _____

TABLE 3: _____

4. WHICH COLUMNS ARE REQUIRED?

5. WHAT CALCULATIONS ARE REQUIRED?

6. WHAT PATTERN DO YOU RECOGNIZE?

- ☐ Filtering
- ☐ Joining
- ☐ Aggregation
- ☐ Group filtering
- ☐ Subquery
- ☐ CTE
- ☐ Ranking
- ☐ Previous/Next row
- ☐ Running total
- ☐ Date comparison
- ☐ Other: _____

7. BREAK THE PROBLEM INTO LAYERS

Layer 1:

Layer 2:

Layer 3:

8. WRITE THE SQL

9. VALIDATE INTERMEDIATE RESULTS

10. VALIDATE FINAL RESULT

20. Trainer Demonstration Method

For your live demo, I strongly recommend this sequence:

Don't do this

Question



Immediately show SQL



Explain SQL

The trainee learns:

"The trainer knows SQL."

But they don't learn how to solve problems.

Check list

QUESTION



ASK TRAINEES:

"What is being asked?"



"One row represents what?"
↓
"Which tables?"
↓
"What calculation?"
↓
"What pattern?"
↓
"What should Layer 1 produce?"
↓
WRITE LAYER 1
↓
SHOW RESULT
↓
"What do we need next?"
↓
WRITE LAYER 2
↓
SHOW RESULT
↓
COMBINE
↓
FINAL QUERY
↓
VALIDATE

21. The Most Important Technique: Hide the SQL

For a demo:

Find customers whose total spending is above the average spending of all customers.

.

Ask:

What is the output grain?

What do we need to calculate?

What SQL functionality gives us that?

`SUM()` + `GROUP BY`.

What does "above average" tell us?

We need the average of customer totals.

Is that the same as the average order amount?

No.

🔥 **This is a critical teaching moment.**

You must understand that:

Average order value

is different from:

Average customer spending

Then construct:

Orders + Order Items

↓

Customer Total

↓

Average Customer Total

↓

Compare

↓

Final Customers

That is genuine SQL reasoning.

22. The Three Questions You Should Ask Repeatedly

If you want trainees to become independent SQL problem solvers, repeatedly ask these three questions:

Question 1

What does one row represent?

This determines:

GROUP BY
PARTITION BY

Question 2

What calculation has to happen first?

This determines your:

Layer 1

Question 3

What has to happen after that?

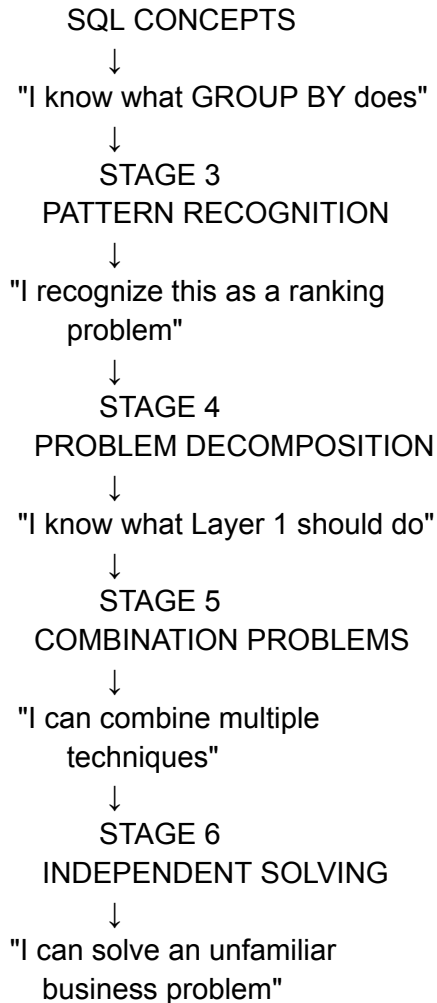
This determines:

Layer 2
Layer 3
CTE
Subquery
Window Function

23. Your Final Teaching Philosophy

The curriculum should gradually move trainees through these stages:

STAGE 1
SQL SYNTAX
↓
"I know SELECT and JOIN"
↓
STAGE 2



The ultimate goal

Don't aim for:

"My trainee knows SQL."

Aim for:

"My trainee can look at an unfamiliar business question, identify the output grain, recognize the SQL pattern, break the problem into logical layers, construct the query, and validate the result."

That is the difference between **SQL syntax training** and **SQL problem-solving training**.

A simple rule to put at the top of every SQL class

This should become the **central philosophy of your SQL training program.**